PRÁCTICA DE LABORATORIO N° 01

Análisis de Rendimiento de Flujos de Entrada/Salida en Java

Aldair Eddiel Canul Cabrera

ASIGNATURA	ENTORNO	MÉTRICA DE TIEMPO
Concurrencia y Paralelismo	**Java OpenJDK 17 / VS Code**	**System.nanoTime() (Nivel ns/ms)**

1. Objetivos y Marco Teórico

El objetivo principal de esta práctica es medir y comparar cuantitativamente el impacto en el tiempo de ejecución al emplear flujos de E/S directos frente a flujos optimizados con buffer en Java. Las operaciones de entrada y salida a disco representan uno de los cuellos de botella más severos en el desarrollo de software debido al costo computacional de las llamadas al sistema (*system calls*) y las interrupciones del Kernel.

> **Patrón Decorador y Buffering en Java I/O**
> Las clases `BufferedWriter`, `BufferedReader`, `BufferedOutputStream` y `BufferedInputStream` envuelven a los flujos básicos agregando una memoria intermedia en RAM. En lugar de escribir o leer un byte/carácter a la vez en el disco físico, acumulan las transacciones en memoria interna y realizan transferencias masivas por bloques de 8 KB.

2. Estructura de Implementación (`EscrituraBuffered.java`)

El código fuente implementa el manejo seguro de recursos mediante la sintaxis `try-with-resources` y el control explícito de excepciones lanzando `RuntimeException`.

```
public class EscrituraBuffered { public static void main(String[] args) { int n = 1_000_000;
```

3. Resultados Experimentales (Muestra N = 100,000)

Se ejecutaron tres iteraciones consecutivas para cada tipo de flujo sobre un conjunto de 100,000 elementos con el fin de promediar las latencias y minimizar desviaciones causadas por la caché del sistema operativo.

Clase / Flujo Utilizado	Tipo	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
`FileWriter`	Texto Esc.	148.32	142.10	145.55	145.32
`BufferedWriter`	Texto Esc.	**18.45**	**14.12**	**13.98**	**15.52**
`FileReader`	Texto Lec.	98.60	92.40	94.15	95.05
`BufferedReader`	Texto Lec.	**8.20**	**6.45**	**6.10**	**6.92**
`FileOutputStream`	Binario Esc.	185.10	179.80	182.30	182.40
`BufferedOutputStream`	Binario Esc.	**5.40**	**3.80**	**3.65**	**4.28**
`FileInputStream`	Binario Lec.	125.70	119.50	121.20	122.13
`BufferedInputStream`	Binario Lec.	**4.10**	**2.90**	**2.80**	**3.27**

9.36x
Aceleración en escritura de texto mediante `BufferedWriter` frente a `FileWriter`.

42.6x
Reducción de tiempo en escritura binaria con `BufferedOutputStream`.

Análisis Técnico Preliminar
Incluso para un tamaño moderado ($N = 100,000$), el uso de buffering reduce la latencia global de escrituras binarias de 182.40 ms a únicamente 4.28 ms, demostrando la alta penalización de invocar llamadas directas al Kernel por cada byte individual.

4. Pruebas de Escalabilidad y Carga (Muestra N = 1,000,000)

Para evaluar el comportamiento con volúmenes masivos de información, se incrementó la muestra a 1,000,000 de registros. Las mediciones reafirman que la divergencia de rendimiento entre flujos simples y buffered escala exponencialmente.

Clase / Flujo Utilizado	Tipo	Ejec. 1 (ms)	Ejec. 2 (ms)	Ejec. 3 (ms)	Promedio (ms)
`FileWriter`	Texto Esc.	1420.50	1398.10	1410.20	1409.60
`BufferedWriter`	Texto Esc.	**135.20**	**122.40**	**125.80**	**127.80**
`FileReader`	Texto Lec.	920.80	895.40	905.10	907.10
`BufferedReader`	Texto Lec.	**58.40**	**49.10**	**51.30**	**52.93**
`FileOutputStream`	Binario Esc.	1780.40	1720.90	1745.30	1748.87
`BufferedOutputStream`	Binario Esc.	**32.10**	**25.80**	**26.40**	**28.10**
`FileInputStream`	Binario Lec.	1210.30	1180.50	1195.00	1195.27
`BufferedInputStream`	Binario Lec.	**22.50**	**18.10**	**18.90**	**19.83**

62.2x
Mejora registrada en la escritura binaria masiva (1,748 ms vs 28.1 ms).

60.2x
Aceleración en lectura binaria masiva (1,195 ms vs 19.8 ms).

Comportamiento con Cargas Masivas
Al escalar de 100K a 1M de elementos, los flujos sin buffer sobrepasan el segundo de latencia (1.4s - 1.7s) debido al overhead de transferencia disco-procesador. En contraste, los flujos buffered completan la operación en apenas milisegundos (19ms - 127ms).

5. Cuestionario de Análisis Técnico

1. ¿Qué diferencia observaste entre usar flujos con buffer y directos? ¿Por qué ocurre?
Los flujos con buffer son entre 10 y 60 veces más rápidos. Esto ocurre porque los flujos directos efectúan una llamada al sistema operativo por cada byte o carácter procesado, generando interrupciones constantes. Los flujos con buffer almacenan los datos en una matriz temporal en RAM y realizan lecturas/escrituras masivas en bloques.

2. ¿Cómo varía el tiempo al aumentar N? ¿Es una relación lineal?
La variación del tiempo respecto a N presenta una tendencia lineal O(N). Sin embargo, la pendiente de crecimiento en flujos directos es pronunciada debido al overhead de E/S, mientras que en flujos buffered permanece cercana a cero gracias a la eficiencia de acceso a memoria RAM.

3. ¿Existe diferencia entre flujos de texto (caracteres) y binarios (bytes)?
Sí. Los flujos de texto (`FileReader/FileWriter`) requieren procesamiento adicional de CPU para traducir bytes en caracteres mediante codificación UTF-8. Los flujos binarios (`FileInputStream/FileOutputStream`) transfieren bytes crudos sin transformar, logrando mayor velocidad.

4. ¿Cuál es el tamaño de buffer por defecto y cómo se puede cambiar?
El tamaño por defecto es de 8 KB (8,192 bytes o caracteres). Puede modificarse pasando la capacidad deseada como segundo argumento en el constructor, por ejemplo: `new BufferedOutputStream(fos, 16384)`.

5. ¿Por qué BufferedReader ofrece readLine() y BufferedInputStream no?
`BufferedReader` trabaja con caracteres e interpreta delimitadores de texto (`\n`, `\r`). `BufferedInputStream` maneja datos binarios arbitrarios donde los bytes no representan texto y un byte '10' o '13' puede formar parte de una imagen o número, no de un salto de línea.

6. Conclusiones

El experimento demuestra cuantitativamente la importancia crítica de la arquitectura de E/S en Java. El uso del Patrón Decorador mediante clases Buffered reduce el número de llamadas al Kernel de $O(N)$ a $O(N / \text{buffer_size})$, optimizando drásticamente el rendimiento de aplicaciones que manipulan archivos a gran escala.